Characterizing the Cornerstone Octree Construction Method

Henry Murdock, William Halpin, Rayan Singh
University of Illinois at Urbana-Champaign
{henrygm3,lhalpin2,rayans2}@illinois.edu

Abstract

An octree is a structure which partitions a 3D space into octants recursively based on positions of objects within the domain. This structure is heavily utilized in scientific simulation, computer graphics, and robotics. A classical use case of an octree is in the field of particle simulations such as those involving short or long range forces where the number of particles required is large enough to become prohibitive for the simulation.

Recent works have expanded upon traditional octree construction methods to allow for distributed simulation on graphics processing units called Locally Essential Trees (LETs). We explore and benchmark a novel method called Cornerstone that implements these methods but also develops a construction method that allows an octree to be rebalanced which is favorable for particle simulations or any problem which frequently changes the positions of the points in some way.

1 Introduction

Scientific simulation, computer graphics, and collision detection as well as other fields utilize structures called octrees. An octree allows a three dimensional space to be broken down hierarchically into a tree structure with up to eight children per node. Meshless point clouds, such as a group of particles, are an important application of such structures, especially when paired with Barnes-Hut or Fast Multipole Methods which allow reductions in the runtime of force calculations between points from $O(n^2)$ to $O(nlog(n))$ and roughly $O(n)$ respectively. This has driven the development of octree construction methods that operate on GPUs and across multiple nodes in clusters and supercomputers.

One of the most interesting new methods for meshless point clouds is Cornerstone [5]. In the case of simulating the movement of particles in dynamic systems, the octree must be recalculated at each time step, however the primary advantage of Cornerstone is to rebalance the tree rather than rebuild from scratch. Cornerstone has been heavily used on exascale supercomputers, however not all problems which may benefit from this method require such systems. Furthermore a profile of the code internals and how the method performs across varying conditions of particle distributions, parameters, and other variants does not appear publicly but may help to optimize and evidence the methods usefulness across varying systems and problem spaces. This project seeks to do such a characterization and will focus closely on the mapping of Cornerstone to the current GPU architectures examining any kernel level metrics of interest, memory movement at the intra and inter node scope including caches, as well as the interactions of communication and computation cost with different algorithm parameters.

2 Cornerstone

To characterize Cornerstone's performance we first need to understand the algorithmic steps. Note that there are two methods of interest, the method where an octree is constructed locally on a node and the case where a locally essential tree is used for distributed simulation. We review the local case first and then advance to the distributed case as an extension. We focus primarily on the code needed and how it maps to GPU architectures, for further insight to the algorithm reference Cornerstone [5]. An outline of the steps in construction is given for the local case.

- Compute a space filling curve (Morton or Hilbert)
- Sort the keys and rearrange any properties associated with the key such as the position, mass, charge, etc
- Update the leaf nodes
- Update the internal tree structure

Additionally Cornerstone has 6 critical parameters which are tabulated below to give an overview. These will also be important parameters for characterization.

Table 1. Cornerstone Parameters

Parameter	Description
Precision	FP64 or FP32
θ	The MAC parameter
N_{crit}	Maximal # of points in a leaf
h	Smoothing parameter
L	The maximal depth of the octree
SFC	Morton or Hilbert

2.1 Space Filling Curves

Informally a space filling curve will take points existing in space and map them to integer keys, when one sorts the original particles based on these keys and traverses the path through them the resulting curve will recursively move through the domain octant by octant (in the case of three

dimensional points). In this way a space filling curve is a kind of fractal.

We are interested in two types of space filling curves namely Morton Z-order curves and Hilbert curves. The path of a Morton Z-order curve may be viewed in figure 1 and shows the distinct z-pattern. Morton codes are computationally efficient to calculate, requiring just a few bit shifts, but as is also evident visually in figure 1 the curve may "jump" octants this may result in unfavorable communication patterns between nodes.

The Hilbert curve on the other hand takes on a compact U shape as seen in figure 2. Hilbert codes offer a "compact" mapping which may reduce the cost of communication but come at the cost in increased computational inefficiency requiring L loop iterations, L being the maximal depth of the octree.

In both cases the calculation is a simple elementwise GPU kernel, which does a relatively small number of operations.

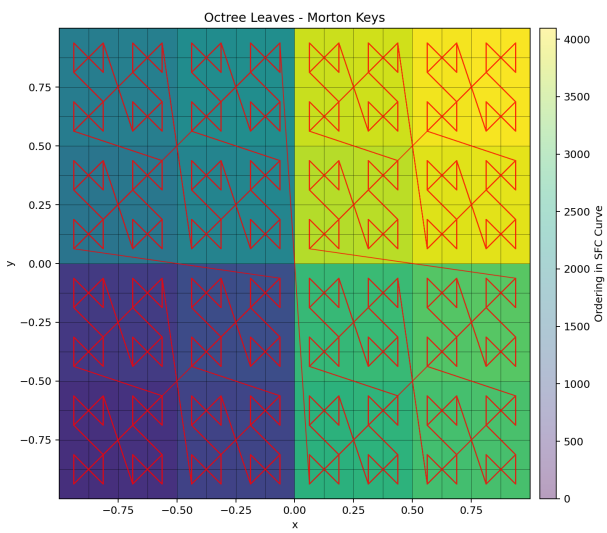

Figure 1. The leaves of an octree colored with respect to their Morton Z-order curve. The red trace plots the curves path through the leaves.

2.2 Radix Sorting and Gathers

We will not detail the Radix sort and Gather implementations as they are well known. The reason these operations are necessary is to sort the keys thus producing the in-order arrangement of keys according to the space filling curve that recursively partitions the space into octants. The keys will have some attributes including x, y, z but may have others based on the application, these also need to be aligned with the sorted keys. In Cornerstone these are distinct kernels with a Radix sort and k gathers for the number of properties

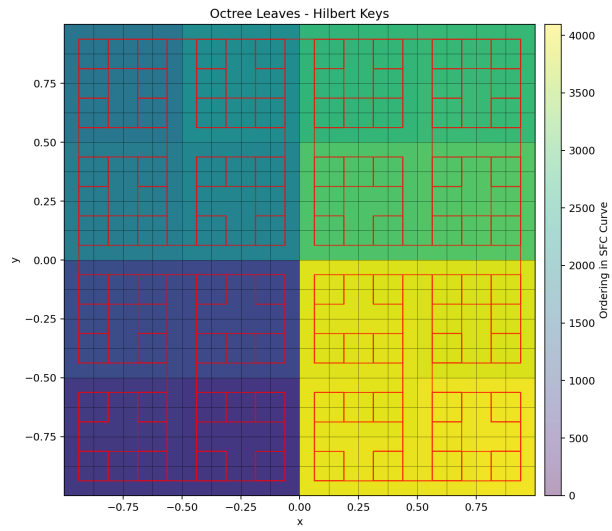

Figure 2. The leaves of an octree colored with respect to their Hilbert curve. The red trace plots the Hilbert curve path through the leaves.

to be sorted. The radix sort is done using the CUB libraries implementation DeviceRadixSort::SortPairs. The gathering of particles in the single node case is left to the user but we used the Thrust libraries gather implementation which has competitive performance.

2.3 Update Leaf Nodes

The namesake of Cornerstone [5] is the Cornerstone array called K. K is a sorted array of SFC keys of length $n_l + 1$, n_l being the number of leaves, with the first element being 0 and the last element being 8^L, with the difference between one element and it's successor being 8^l, where $l \leq L$. Then K will define a unique octree but represents explicitly the leaves. Cornerstone [5] also uses an N_{crit} parameter which defines the maximum allowable number of particles a leaf node may have; in other words this parameter defines the resolution of the octree or depth at which it must be resolved. Then the steps to generate the Cornerstone array from the particle SFCs are to calculate a histogram of the particle SFCs belonging to each each leaf node and then perform three steps:

1. Compute the rebalancing decision for each node i.e a code where 8 indicates $N_{crit} < N_i$ and so the node should be subdivided, 0 indicates the node should be replaced, and 1 indicates the leaf should be unchanged.
2. Perform an exclusive prefix sum of the array of computed rebalancing decision codes, this leads to each element in the result being the new index of the node in the corresponding new Cornerstone array. The final

element is the number of nodes in the new Cornerstone array.
3. Finally, if the rebalancing decision for node i is 1 then the key is copied to the new index calculated in the exclusive prefix sum. If the decision is to split, the code is 8 and we copy the keys of the children of the original node to the new range.

A Cornerstone array is then balanced once no node needs to be split or merged, which means that every leaf node has a particle count $\leq N_{crit}$ and every internal node has a particle count $> N_{crit}$. By iteratively applying these steps until we reach the criteria for a balanced Cornerstone array we define a unique balanced octree.

Regarding performance, note that each kernel launch here must occur in serial, no stream parallelism is possible as global synchronization is necessary between each step, however there are some techniques that may allow kernel fusion to be effective which may be of interest depending on the high level bottlenecks. The calculation of the rebalancing decision is an elementwise kernel and in practice may use larger multiples of 8 to indicate multiple levels need to be merged. The exclusive prefix sum Additionally, a memory allocation is necessary and has to read the value of the last element of the exclusive prefix sum to calculate the number of new leaf nodes.

2.4 Update Internal Nodes

Finally the internal tree structure must be created allowing us to traverse the tree and ultimately we are left with three arrays of integers, note $n = n_i + n_l$ with n_i being the number of internal nodes and n_l being the number of leaf nodes:

1. NK of size n, with each value being within the range defined by the SFC size supported by L which is $[0, 2^{3L})$. This is the SFC key for each node both leaves and internal nodes, note that because the SFC can decode to 3D points this also tells us the 3D positions of each node.
2. CO of size n, with each value being in the range $[1, n)$. This is the connectivity array.
3. LO of size $L+2$ with each value being in the range $[1, n)$ which are the tree level offsets, that is the j element of LO gives the index into CO and NK that is the first node in the level j of the tree.

This requires several operations, the first is allocating NK and copying the Cornerstone array to the portion of the range for leaf nodes, this is an elementwise kernel. Then the algorithm computes another elementwise kernel that does an xor of two adjacent keys in the Cornerstone array and then counts the leading zeros. Then NK is sorted via radix sort which gives a breadth-first node ordering, again using the CUB implementations. Then the level offsets LO are generated with a binary search of NK, another elementwise kernel. And finally the connectivity array CO is generated by doing a binary search for the first child of a node using the level offsets to reduce the problem size.

An important performance note here that will show up in 4 is that the computation of SFC keys and the subsequent radix sort and gathers all operate on n_p which is the number of bodies in simulation space which is much larger than the computation being done in the later two steps where we operate only on the leaf nodes assuming N_{crit} is sufficiently large. We discuss in further detail in 4.

2.5 Domain Decomposition

The above process outlines the process to build a balanced octree on a single rank. This method can be extended however to allow multiple peers to decompose the domain among themselves, this allows a rank to maintain a well resolved local domain with a coarser global domain. This modifies the process of generating the octree to compute a global octree and a focused tree.

The global octree allows us to decompose the domain of particles among the peer ranks based on the SFC, calculating the coarse global domain with a higher N_{crit} called $N_{crit,glob}$. This process modifies the above algorithm so that the initial histogram step in the Cornerstone array rebalance does a global sum of the histograms from each rank . Then performing the balancing steps using the histogram of the global domain rather than the local domain yields a balanced global octree. Such an algorithm would incur unacceptable communication costs at high resolution (lower N_{crit}) as it involves an all-to-all communication which would be iterated on; more resolved domains requiring more iterations to balance the tree. However at a coarser resolution we can use this to efficiently decompose the global domain by partitioning the Cornerstone arrays space filling curve among the ranks the collective communication of this method is $O(nlogn)$ where n is the total number of particles across all ranks.

2.6 Locally Essential Trees

Once the space filling curve is decomposed among n_{ranks} subdomains the algorithm attempts to find a Locally Essential Tree which includes the nodes necessary for computation within the local subdomain F. Here Cornerstone [5] incorporates physical context into the algorithm considering both short-range forces and long-range forces. Both require a high resolution with F and at the exterior surface of F for halo particle discovery. Long-range forces are assumed to be computed via Barnes-Hut or the Fast Multipole Method both of which use a Multipole Acceptance Criterion (MAC) function to determine whether the multipole approximation applies given a source node and target point or node; the multipole approximation being that the force can be computed by using the source node as a kind of super particle that acts on the target with a force relative to the number of particles within it. The most common MAC, where r_{min} is the minimum distance between source and target, l is the source

node edge length, and θ is a controllable accuracy parameter as noted in the table above, is of the form $r_{min} > l/\theta$, if this inequality is true then the the multipole approximation is valid. Note that as the θ parameter is increased the multipole approximation is accepted at smaller distances or for larger source node lengths, this tends to make the solution less accurate as the approximation usually becomes less valid for nodes that are larger or closer to the target.

Finally this results in several new arrays and functions to balance this Locally Essential Tree. We also represent a LET with a Cornerstone array and construction remains similar with some tweaks. First we compute whether whether the multipole approximation is accepted for every node outside of F, the multipole approximation is marked as not acceptable if it fails for any target node in F, the array of evaluations is called M. Some MACs may require communication between ranks. Rebalancing is now additionally a function of M and does not require communication while generating the histogram of node counts requires communication. The bottleneck according to Keller et al. [5] is the function calculates the node properties based on particles and the nodes children which is done via an upward pass using point-to-point communication for particles outside of F but where the LET has finer resolution than the global tree (category 2) and collective communication for particles outside of F in nodes where the resolution of the global tree exceeds that of the LET; note that the LET has a variable resolution outside of F based on the MAC. The overall process is as follows then:

1. First the node properties for leaves in F are computed locally (category 1). Each rank finds category 2 nodes by traversing the global octree and evaluating as stated above.
2. Ranks send p2p messages of their octree nodes to peer ranks. The receiver uses the nodes to fill in properties for category 2 nodes and then the node properties are sent back to the requesting peer.
3. Perform an *MPI_Allgatherv* communication to all nodes to send the aggregated LET properties across the ranks, then perform another upward pass on each node to get the properties all the way up to the global root node.

This yields a focused LET at each rank allowing traversal of the local subdomain and the calculation of forces.

3 Cornerstone Analysis

The goal of this project is an analysis of Cornerstone [5] across varying parameter spaces, how the problem maps onto GPU hardware, in the distributed case the communication bottlenecks imposed by the algorithm, and to take advantage of any optimization opportunities that arise. There may be multiple opportunities for improvement as Cornerstone [5] appears to be mostly optimized for the distributed case. Much of the already existing performance analysis in regards to Cornerstone has been for extremely large particle counts with computation distributed across thousands of GPUs. For smaller application spaces such as real-time rendering or simulations on orders of magnitude around or less than 10^8, a single GPU or a few is likely more optimal due to the constraints imposed by communication costs, against the processing power of the a single GPU, as well as the significant reduction in work that the local case allows. As we will see the overhead for distributing work is not trivial. Due to technical constraints, our preliminary results and analysis operate using only a single node with with 1 or 2 peer ranks each with their own GPU. However these type of workloads involving only modest numbers of GPUs are not uncommon especially in applications like computer aided engineering, and analysis on these workloads may yield useful conclusions for a number of use cases.

3.1 Code

To evaluate the Cornerstone octree construction method under realistic conditions, we developed a configurable batch generation pipeline that sweeps over a matrix of particle distributions, particle counts, and perturbation magnitudes. This hooks into our testing harness that runs the simulation with the ability to select between using GPUs or CPUs, the local only or distributed case using a global tree and focus tree, and other parameters of interest. Particles that leave the domain after perturbation are truncated back to the boundary. We have created the necessary Jupyter notebooks and Python scripts to generate distributions of interest and their perturbations. This includes code to save those distributions to HDF5 files. We developed a framework in C++ for loading those HDF5 distributions and their perturbations and generating the corresponding octrees, adding the perturbations to the particles and then rebalancing the octree. This allows us to observe the effects of the initial construction and across varying magnitudes of perturbations. This includes methods for a CPU and GPU implementation as well as their related distributed MPI variants. This is flexible enough to allow us to generate any metrics of interest in the future and has already allowed us to gather the results discussed in 4. Tooling to instrument sections of interest is done using NVIDIA Tooling Extensions (NVTX). We instrumented our code to highlight important metrics at a relatively coarse level so that we could observe the primary steps necessary to building the Octree and focus our attention on bottlenecks.

3.2 Distribution Generators

All initial particle distributions are confined to the cube domain $[-1, 1]^3$, where all distributions are as large as the domain. Six initial particle distributions are used:

1. **`uniform_initial`**: Samples particles uniformly at random from the domain.

2. **pancake_initial**: A cylindrical distribution along the Z-axis where the Z-axis is strongly compressed, as to resemble a disk, frisbee, or (as the name suggests) pancake.
3. **pancake_initial_tilted**: Same as the pancake but tilted off-axis, so the thin dimension is no longer aligned with a coordinate plane.
4. **filament_z_initial**: A one-dimensional filament aligned along the z-axis, characterized by the parametric equation: (0, 0, t).
5. **filament_yz_initial**: A filament lying in the yz-plane, anisotropic in two directions, characterized by the parametric equation: (0, t, t)..
6. **filament_xyz_initial**: A filament oriented along the domain diagonal, anisotropic in all three coordinate directions simultaneously, characterized by the parametric equation: (t, t, t).

By analyzing 1D, 2D, and 3D manifolds that are or are not axis-aligned, a wide variety of behavior can be observed after applying perturbations. Because the perturbation distribution is 3-dimensional, the various distributions will have their unique attributes degraded as the perturbation magnitude increases.

3.3 Perturbation Model

After the initial distribution is generated, a perturbation vector is applied to simulate particle motion between timesteps. The perturbation function draws each displacement component independently from $\mathcal{U}(-s, +s)$ where s is the scale factor. Particles that move outside the domain after perturbation are truncated to the boundary (`out_of_bounds = 'truncate'`). Four perturbation scales are used:

- $s = 0.001$: Very small perturbation; particles barely move relative to the domain.
- $s = 0.01$: Moderate perturbation; noticeable displacement but locally coherent.
- $s = 0.1$: Large perturbation; significant displacement triggering substantial tree restructuring.
- $s = 0.5$: Very large perturbation; particles may traverse a large fraction of the domain.

3.4 Runtime Configuration

The Cornerstone framework accepts several runtime parameters via the command line:

- `-bucket-size-global`: Maximum number of particles per leaf node in the global octree (default: 1024).
- `-bucket-size-focus`: Maximum number of particles per leaf node in the focused octree (default: 64).
- `-theta`: The multipole acceptance criterion (MAC) parameter controlling the accuracy/performance tradeoff (default: 0.6).
- `-gpu`: Flag to enable GPU-accelerated octree construction.
- `-lets`: Flag to enable Cornerstone [5] to use Locally Essential Trees and communicate with peer ranks rather than use only local trees.

No rotations were applied (`rotations = None`).

3.5 Test Configurations

The full sweep comprises 6 distributions × 5 particle counts × 4 perturbation scales = 120 configurations. There were also several tests run independently of this parameter sweep to gather general metrics which used a uniform distribution of particles on the unit cube.

3.6 Machines

For the results used here we have used CloudLab to gain access to GPUs allowing us to get on a node with 4 V100s; however, access to this is difficult to attain on a regular basis and does not provide access to bare metal. We have access to the NCSA Delta cluster at this point and will use it to gather metrics on other GPUs and in varying configurations during the second half of the project. This should also provide much more consistent access to the resources we need. The Delta cluster comes with 4x and 8x GPU configurations per node with A40, A100, and H200 GPUs being the devices we are interested in testing on in the future. We expect that we will not do inter-node experiments as it is outside of the hardware limits we have.

For all the results presented in 4 we used a c4130 node on the CloudLab Wisconsin cluster. The node had four NVIDIA Tesla V100 GPUs. Each GPU had 16GB of memory in an SXM2 configuration with 900 GB/sec of peak memory bandwidth. A single V100 has 84 Streaming multiprocessors each with 64 FP32 cores, 64 INT32 cores, and 32 FP64 cores.

4 Results

We used our framework to perform several experiments. We explored several questions for the midterm the first was parameter spaces we tests over different distributions, for both Morton and Hilbert keys, and the initial build versus the subsequent perturbation rebalance.

A general test to develop a sense of Cornerstone's internal bottlenecks is done on a single GPU comparing a case with 100 million using either Hilbert or Morton keys. The test loop contains the code to compute the SFC, do a radix sort, reorder the corresponding particle properties, and update the leaves and then internal tree state; these are the steps needed to compute the octree and have it in a usable state. Immediately we see that the sorting and reordering of particle properties is very dominant in this regime. Therefore, these kernels will be the main target of optimizations, which will be covered in the latter stages of this project.

Figure 6 breaks down the runtime of both the Hilbert and Morton key cases using the local only build with 100 million particles. The dominant factor in the calculation is

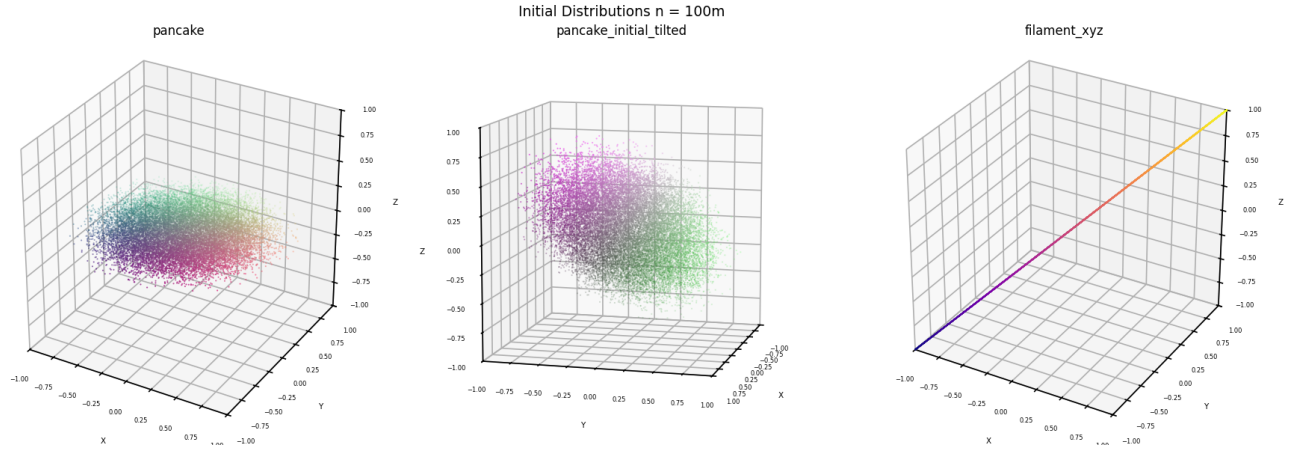

Figure 3. Initial particle distributions with 10^8 particles.

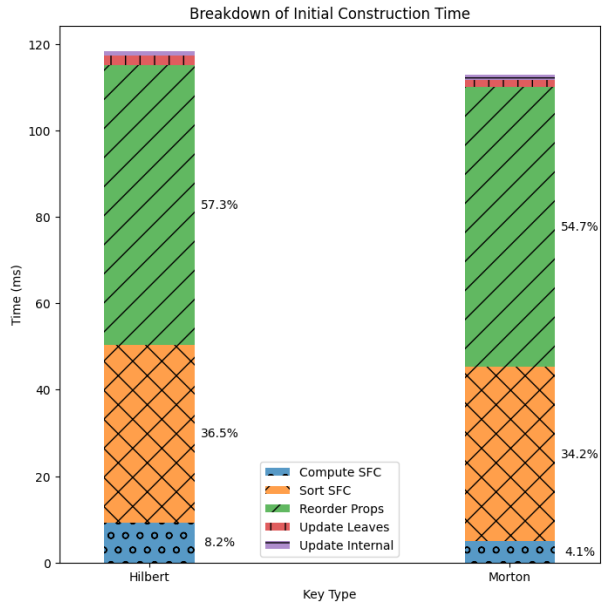

Figure 4. Comparison of operations in a broader tree update on a single GPU.

the reordering and sorting of the keys and corresponding particles with the actual building of the tree and computation of the space filling curves taking only ≈ 10% of the total build time.

Focusing on overall runtime, there is a stark distinction of behavior between low particle count (1 million and less) and high particle count (10 million and more) workloads. The runtime of low particle workloads either is not significantly impacted by the initial distribution, as shown in Figure 4 or behaves unpredictably as shown in Figure 5. A consistent pattern emerges across high particle workloads, which is shown in Figure 6. Here we can see that, generally, lower dimension manifolds have lower construction cost associated with them.

It should be noted that the construction time does not scale linearly with the number of particles at low particle counts as gleamed from figures 7, 8, and 9. As cornerstone has been largely been tuned for very high particle count workloads, it is not surprising that low particle count workloads do not have the same performance characteristics.

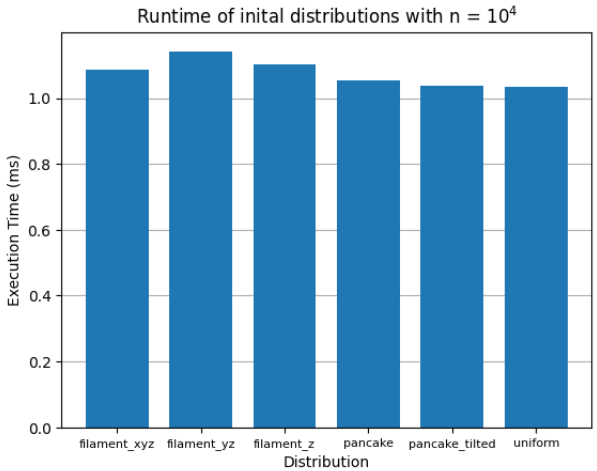

Figure 5. Construction time of different initial distributions for a low particle workload of n = 10^4.

To analyze the effects of perturbation, the update time of the octree after a perturbation is applied was compared to the initial construction time octree for each distribution. It is clear that, for the low particle workloads, perturbations have very little impact on update time as shown in Figure 10. Only for high particle workloads does the correlation of perturbation strength to relative update speed become noticeable, as shown in Figure 11.

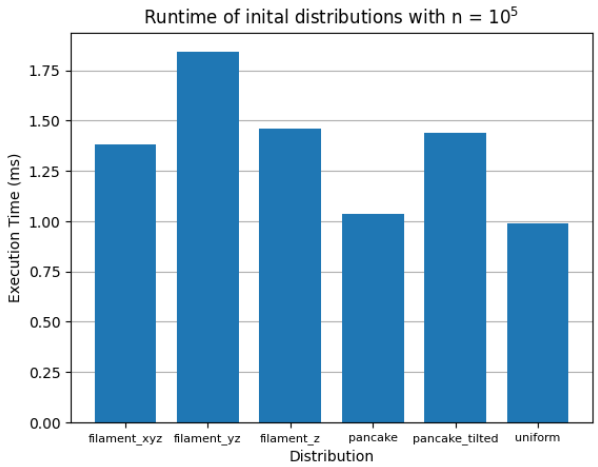

Figure 6. Construction time of different initial distributions for a low particle workload of n = 10^5.

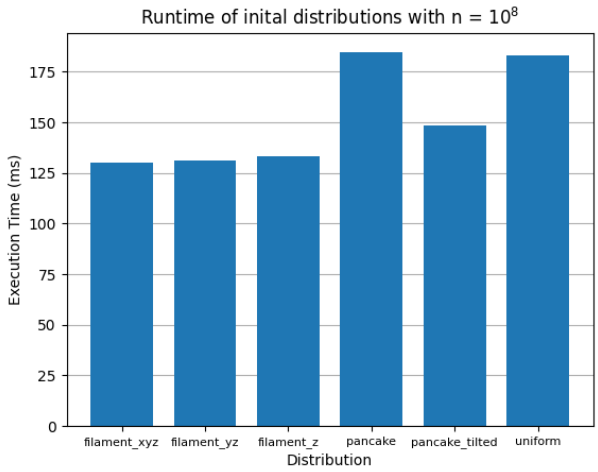

Figure 7. Construction time of different initial distributions for a high particle workload of n = 10^8.

The perturbations have varying effectiveness depending on the initial distribution to which they are applied. This is likely due to the fact that the initial distributions are easily expressed by the octree, which applying a perturbation degrades. This is bolstered by the fact that the distributions which are most susceptible to perturbations are also the one's that have the lowest initial construction time. This is likely why the uniform distribution is not affected greatly by perturbations until a perturbation level of .5, when the filament distributions see a steady performance decrease at increasing perturbation levels. It is important to note that a relative execution time greater than 100% does not mean updating the octree is slower than reconstructing it from scratch as the perturbed distribution may still take longer to generate from scratch.

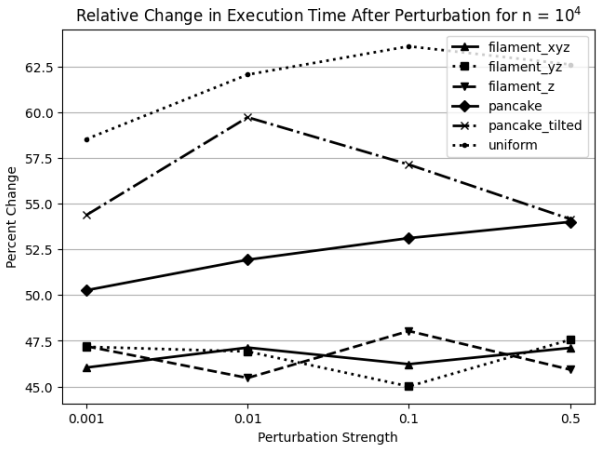

Figure 8. Perturbed octree rebalancing time relative to initial build time for different perturbation strengths for a low particle workload of n = 10^4.

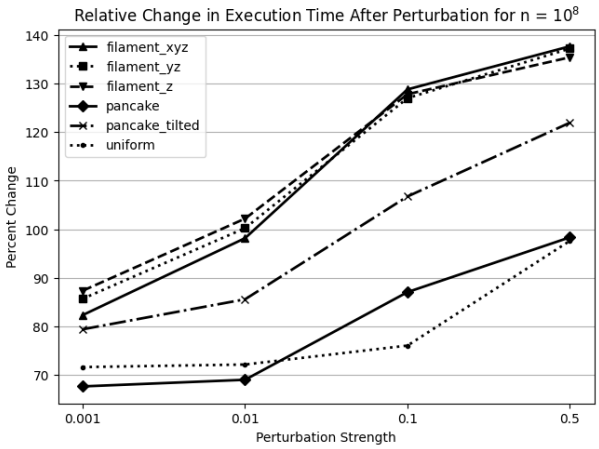

Figure 9. Perturbed octree rebalancing time relative to initial build time for different perturbation strengths for a high particle workload of n = 10^8.

Overall in figure 11 the results were as expected. The general trend shows the rebalancing of the perturbed octree has a dependency on the strength of perturbations. That is as perturbation strength grows time to rebalance grows. What we did not expect to see was that the rebalancing would become significantly greater than even the original build timings, up to ≈ 40% in some cases. This will need to be explored at greater depth in the second half of the project.

Figure 12 shows the difference in timing in local octree build times vs. the corresponding times for building the distributed domain for the same number of particles. This shows us the overhead of the distributed case over the single node case. Clearly the overhead is significant at approximately two orders of magnitude in this case. On observation of the

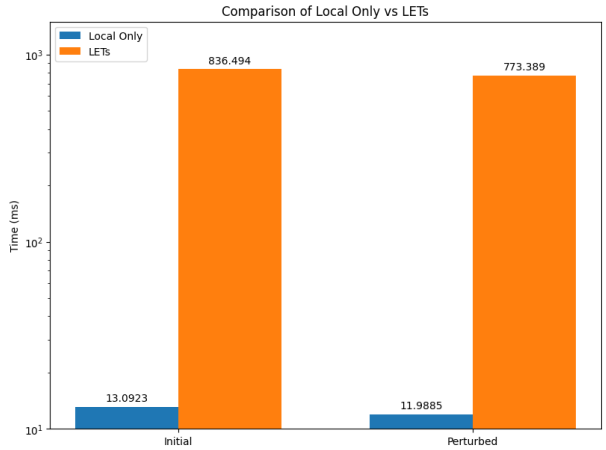

Figure 10. Comparison of locally essential tree build time vs. local only using 10 million particles, Hilbert keys, and a uniform distribution using 2 ranks, each with their own GPU.

profile the largest time sync is the calculation of peer ranks according to the multipole acceptance criterion.

5 Remaining Work

There is still significant potential to expand the breadth of our project over the second half of the semester. So far we have tested mostly uniform perturbations we plan to augment this to test many different types of affine transforms that the method may encounter such as rotations and translations. We also want to look at domain over-sizing, the domain bounding box discussed previously defines the region in which to begin subdividing into octants. If this box is significantly larger than the true domain of the particles it will cause the octree to have large regions of empty space which requires the octree to become deeper thus inducing inefficiencies. A study on the sizing of the bounding box against the build time should be conducted. An interesting parameter to study would be L which is the maximal depth an octree can be resolved to. The two values usually used are 32-bit integers which allow resolution to a depth of 10 levels and 64-bit integers which allow resolution to a depth of 21. The tradeoff can sacrifice some accuracy but can reduce the amount of memory movement significantly. This is especially of interest since the radix sort and gather kernels are bottlenecks in the local case both of which are memory bound kernels which can be improved by reducing global memory pressure.

We have mostly looked at NSight Systems results to understand communication bottlenecks and which sections of code are the largest bottlenecks, assuming it is of interest we will likely look at some NSight Compute results for the kernels to determine specifically which instructions are causing issues. The radix sort and gather kernels are both from the NVIDIA Core Compute Library (CCCL) and should be very optimized, so optimizing the kernel code is unlikely to provide significant speedup. The idea of kernel fusion may provide significant improvements by generating the sorted index and then reordering all of the particle properties at once without having to writeback the indexes or keys to memory and then load them again. This would reduce the number of memory operations required and the get rid of the overhead of multiple kernel launches. Another idea would be to use stream based execution although this would likely be less optimal than kernel fusion as it only parallelizes the gather operations rather than truly reducing the amount of memory operations. These two studies would likely be interesting and both options may be explored in the final report.

6 Related Work

Karras [4] developed a fully parallel binary radix tree construction method for the GPU. Cornerstone builds directly on these algorithms, extending them from single-GPU graphics to distributed with locally essential trees (LETs) and iterative rebalancing. Apetrei [1] improved on Karras by fusing bounding box calculation into a single bottom-up kernel pass. Cornerstone also minimizes kernel launches but operates on the leaf-level SFC key array rather than an explicit node hierarchy, helping accelerate rebalancing.

Warren and Salmon [9] introduced the parallel hashed octree, replacing pointer-based structures with hash tables for distributed N-body simulation. Cornerstone shares the distributed octree goal but replaces the hash scheme with a linear SFC-key representation that maps naturally onto GPU scan kernels.

Burtscher and Pingali [3] presented the first complete GPU-only Barnes-Hut implementation, showing 74x speedup over optimized serial CPU code. Unlike Cornerstone, their tree is rebuilt from scratch each timestep and targets only the single-GPU case. Bédorf et al. [2] developed Bonsai, a GPU-resident sparse octree treecode processing over 2.8M particles/sec with 20x speedup over CPU. Cornerstone follows the same GPU-resident philosophy but adds distributed multi-GPU support and rebalancing rather than full reconstruction.

Sundar et al. [7] presented parallel bottom-up construction and 2:1 balance refinement of linear octrees, handling over a billion octants in under a minute on 1024 CPUs. While both DENDRO and Cornerstone target scalable octree construction, DENDRO is designed for CPU-based adaptive mesh PDE solvers, while Cornerstone targets GPU particle simulations with incremental updating.

Lauterbach et al. [6] compared Morton-code LBVH builds against SAH approaches for GPU BVH construction, noting the construction speed versus traversal quality tradeoff.

Cornerstone uses a similar Morton encoding but targets particle simulations and allows Hilbert curve usage. Deng et al. [?] introduced the G-ML-Octree, showing an order of magnitude update cost reduction for moving 3D objects on GPUs through a loose octree with memoized updates. Both G-ML-Octree and Cornerstone avoid full reconstruction, but Cornerstone achieves this by iteratively adjusting the SFC key boundaries of its leaf array instead of maintaining a loose hierarchy with memoized updates.

7 Conclusion

In summary we have presented our initial efforts towards analyzing Cornerstone [5]. The Cornerstone [5] code and ours was instrumented using the NVTX profiling tools. We developed a harness for running the application using GPUs or CPUs and with either a local octree setup or a distributed setup. Around this we created scripts to run parameter sweeps and gather profiles and the octree view. We also developed scripts to generate the distributions of initial positions and their perturbations. And we detailed the kernels and code that are required by the algorithm.

To gather initial results we used a CloudLab node with 4 V100 GPUs. We profiled the local octree case and a case with 2 MPI ranks each with their own V100 GPU. We showed that perturbation strength did correlate with the rebalancing time but only with larger numbers of particles. The sorting and reordering steps dominate the computation of the tree locally. We also showed that the distributed case introduced significant overhead near two orders of magnitude but was not due to communication costs which needs further exploration.

There are more kernel launches and memory operations inside of Cornerstone than was expected, and it may be of interest to bring in a baseline comparison in the second half of the semester. Some methods have very few kernel launches like Apetrei [1] and Karras [4]. However $\approx$ 95% of the bottleneck is still on the computation of SFCs, radix sort, and gather operations all of which are still necessary in those methods so we will likely optimize for this and then revisit whether a baseline comparison is necessary.

8 Metadata

The code/data of the project can be found at:

https://github.com/Murdock022X/pca533-octrees

References

[1] Apetrei, C. Fast and Simple Agglomerative LBVH Construction. In *Computer Graphics and Visual Computing (CGVC)* (2014), R. Borgo and W. Tang, Eds., The Eurographics Association.

[2] Bédorf, J., Gaburov, E., and Portegies Zwart, S. A sparse octree gravitational n-body code that runs entirely on the GPU processor. *Journal of Computational Physics 231*, 7 (2012), 2825–2839.

[3] Burtscher, M., and Pingali, K. An efficient CUDA implementation of the tree-based Barnes Hut n-body algorithm. In *GPU Computing Gems Emerald Edition*. Morgan Kaufmann, 2011, ch. 6, pp. 75–92.

[4] Karras, T. Maximizing Parallelism in the Construction of BVHs, Octrees, and k-d Trees. In *Eurographics/ ACM SIGGRAPH Symposium on High Performance Graphics* (2012), C. Dachsbacher, J. Munkberg, and J. Pantaleoni, Eds., The Eurographics Association.

[5] Keller, S., Cavelan, A., Cabezon, R., Mayer, L., and Ciorba, F. Cornerstone: Octree construction algorithms for scalable particle simulations. In *Proceedings of the Platform for Advanced Scientific Computing Conference* (New York, NY, USA, 2023), PASC '23, Association for Computing Machinery.

[6] Lauterbach, C., Garland, M., Sengupta, S., Luebke, D., and Manocha, D. Fast BVH construction on GPUs. *Computer Graphics Forum 28*, 2 (2009), 375–384.

[7] Sundar, H., Sampath, R. S., and Biros, G. Bottom-up construction and 2:1 balance refinement of linear octrees in parallel. *SIAM Journal on Scientific Computing 30*, 5 (2008), 2675–2708.

[8] Wang, M., et al. G-ML-Octree: An update-efficient index structure for simulating 3D moving objects across GPUs. *Newcastle University ePrints* (2022).

[9] Warren, M. S., and Salmon, J. K. A parallel hashed oct-tree n-body algorithm. In *Proceedings of Supercomputing '93* (1993), ACM, pp. 12–21.